



FACULTAD DE INGENIERÍA

CARRERA DE INGENIERÍA DE SISTEMAS
COMPUTACIONALES

INFORME DE PROYECTO FINAL

**Sistema de planificación de inversiones con presupuesto
limitado**

Autores:

NICOLAI STEFFANO LOPEZ ENCISO (100%)
YOSMEL MALVACEDA VELASQUEZ (100%)
VICTOR ALFREDO MARQUEZ DAVILA (100%)
FABIO SEBASTIAN PEREDA LISBOA (100%)
JOSE FABIAN ROJAS DEL HIERRO (100%)

Curso:

Análisis de Algoritmos y Estrategias de Programación

Docente del Curso:

MARCELINO TORRES VILLANUEVA

Lima – Perú

2026-1

Contenido

RESUMEN.....	3
I. INTRODUCCIÓN.....	3
1.1. Definición de objetivos	4
1.2. Descripción del problema	4
II. ANÁLISIS DE LA SOLUCIÓN	5
2.1. Elección de la solución.....	5
2.2. Herramientas de Ingeniería.....	5
III. DESARROLLO DE LA SOLUCIÓN	6
3.1. Formulación del Pseudocódigo	6
3.2. Implementación del Algoritmo	7
IV. RESULTADOS	8
4.1. Análisis empírico	8
4.2. Evaluación	8
V. CONCLUSIONES.....	9
VIII. REFERENCIAS O BIBLIOGRAFÍA	9
IX. ANEXOS	9

INDICE DE TABLAS
INDICE DE FIGURAS
INDICE DE ANEXOS

RESUMEN.

La planificación de inversiones representa un desafío importante cuando se dispone de un presupuesto limitado y múltiples opciones de inversión con diferentes costos y beneficios esperados. Para este caso se debe seleccionar la mejor combinación de inversiones porque requiere el uso de técnicas algorítmicas para permitir la optimización de los recursos disponibles y así maximizar la rentabilidad obtenida.

En el proyecto que hemos desarrollado hicimos un sistema de planificación de inversiones implementado en el lenguaje de programación Python, utilizando dos estrategias algorítmicas que son el algoritmo Voraz (Greedy) y la Programación Dinámica mediante el problema clásico de la Mochila 0/1. Este sistema permite registrar un conjunto de inversiones, ingresar un presupuesto disponible y determinar automáticamente cuáles inversiones deben seleccionarse para así obtener la mayor ganancia posible sin exceder el presupuesto que se establezca.

Además, realizaremos una comparación entre las dos estrategias para considerar la calidad de la solución obtenida, su tiempo de ejecución y la complejidad computacional. Al final mediante esas pruebas determinaremos la programación dinámica garantiza una solución mas optima, mientras que el algoritmo voraz ofrecerá tiempos de respuestas menores, pero no garantiza encontrar la mejor combinación de inversiones.

Palabras clave: Programación Dinámica, Mochila 0/1, Algoritmo Voraz

Investment planning poses a significant challenge when faced with a limited budget and multiple investment options, each with varying costs and expected benefits. Selecting the optimal combination of investments requires algorithmic techniques to optimize available resources and maximize returns.

We developed an investment planning system using Python that employs two algorithmic strategies: the Greedy algorithm and Dynamic Programming (specifically, the classic 0/1 Knapsack problem). The system allows users to input a set of investments and an available budget, then automatically determines the best selection of investments to achieve the highest possible return without exceeding the set budget.

Furthermore, we compare the two strategies based on solution quality, execution time, and computational complexity. The tests demonstrate that Dynamic Programming guarantees a more optimal solution, whereas the Greedy algorithm offers faster response times but does not guarantee finding the best investment combination.

Keywords: Dynamic Programming, 0/1 Knapsack, Greedy Algorithm

I. INTRODUCCIÓN.

1.1. Definición de objetivos

Objetivo general

Desarrollamos un sistema de planificación de inversiones con presupuesto limitado mediante la implementación de los algoritmos de Voraz y Programación Dinámica (Mochila 0/1), para así determinar la combinación de inversiones que maximice la ganancia sin exceder el presupuesto disponible.

Objetivos específicos

- Analizar el problema de planificación de inversiones identificando sus entradas, salidas, restricciones y criterios de solución.
- Diseñar una solución computacional utilizando Programación Orientada a Objetos, funciones y estructuras de datos adecuadas.
- Implementar el algoritmo Voraz para seleccionar inversiones considerando la mejor relación entre ganancia y costo.
- Implementar el algoritmo de Programación Dinámica basado en la Mochila 0/1 para obtener la solución óptima del problema.
- Comparar el rendimiento y la eficiencia de ambos algoritmos mediante pruebas experimentales y análisis de complejidad temporal y espacial.
- Validar el funcionamiento del sistema utilizando diferentes casos de prueba que permitan comprobar la correcta selección de inversiones.

1.2. Descripción del problema

En la actualidad existen diversas oportunidades de inversión, como acciones, bonos, fondos mutuos, criptomonedas y proyectos empresariales. Cada una de estas alternativas requiere una determinada cantidad de dinero y ofrece una ganancia esperada diferente.

Uno de los principales problemas para cualquier inversionista consiste en decidir cuáles inversiones realizar cuando el presupuesto disponible no es suficiente para cubrir todas las opciones existentes. Una mala elección puede ocasionar una disminución considerable en las ganancias obtenidas o el desaprovechamiento del presupuesto disponible.

Por ejemplo, un inversionista dispone de S/ 1,000 y tiene cinco oportunidades de inversión. Algunas ofrecen una mayor rentabilidad, mientras que otras requieren una inversión más elevada. Si el inversionista selecciona únicamente las inversiones con mayor ganancia individual, podría desperdiciar parte del presupuesto y obtener una ganancia total menor que la que podría conseguir mediante una mejor combinación de inversiones.

Desde el punto de vista algorítmico, este problema puede modelarse como una variante del problema clásico de la Mochila 0/1, donde:

- El presupuesto representa la capacidad máxima de la mochila.
- El costo de cada inversión representa el peso de cada objeto.
- La ganancia esperada representa el valor o beneficio del objeto.
- Cada inversión puede seleccionarse una única vez.

Con el propósito de resolver esta problemática, se implementan dos enfoques diferentes. El primero corresponde al algoritmo Voraz, que selecciona inicialmente las inversiones con mejor relación beneficio/costo. El segundo utiliza Programación Dinámica mediante el algoritmo de la Mochila 0/1, el cual evalúa múltiples combinaciones para garantizar la obtención de la solución óptima.

El desarrollo de este proyecto permite demostrar la importancia de seleccionar adecuadamente la estrategia algorítmica según las características del problema, comparando tanto la calidad de la solución obtenida como la eficiencia computacional de cada algoritmo.

II. ANÁLISIS DE LA SOLUCIÓN

2.1. Elección de la solución

Para este proyecto hemos decidido usar el algoritmo Voraz (Greedy) y la programación Dinámica mediante el algoritmo de la 0/1.

Justifica las ventajas y desventajas del algoritmo implementado en una tabla.

Algoritmo: Voraz (Greedy)	
Ventajas	Desventajas
-Fácil de implementar -Tiempo de ejecución reducido -Consume poca memoria -Adecuado para problemas grandes donde se requiere rapidez	-No garantiza la solución óptima -Puede desperdiciar parte del presupuesto -Solo considera decisiones locales -No evalúa todas las combinaciones posibles.

Algoritmo: Programación Dinámica (Mochila 0/1)	
Ventajas	Desventajas
-Siempre obtiene la mejor solución posible -Evalúa todas las combinaciones necesarias -Ideal para problemas de optimización -Garantiza el máximo beneficio	-Mayor tiempo de ejecución -Consume mayor cantidad de memoria -La implementación es más compleja -Su rendimiento disminuye cuando el presupuesto es muy grande

2.2. Herramientas de Ingeniería

<i>Python</i>	<i>Lenguaje principal para implementar algoritmos debido a su sintaxis sencilla y fácil de manejar</i>
<i>Visual Studio Code</i>	<i>Editor de código que se usa para escribir, ejecutar y depurar el programa</i>
<i>Git</i>	<i>Usamos este sistema para el control de versiones para registrarlos</i>
<i>GitHub</i>	<i>La utilizamos para el fácil uso en equipo</i>
<i>Word</i>	<i>Lo usamos para elaborar el informe</i>

III. DESARROLLO DE LA SOLUCIÓN

3.1. Formulación del Pseudocódigo

Algoritmo voraz:

Inicio

Leer presupuesto

Leer cantidad de inversiones

Para cada inversión hacer

Leer nombre

Leer costo

Leer ganancia

Fin Para

Ordenar las inversiones de mayor a menor según la relación:

Ganancia / Costo

costoTotal ← 0

gananciaTotal ← 0

Para cada inversión ordenada hacer

Si costoTotal + costo ≤ presupuesto Entonces

Agregar inversión a la lista seleccionada

costoTotal ← *costoTotal* + *costo*

gananciaTotal ← *gananciaTotal* + *ganancia*

Fin Si

Fin Para

Mostrar inversiones seleccionadas

Mostrar costo total

Mostrar ganancia total

Mostrar presupuesto restante

Fin

Algoritmo programación dinámica

Inicio

Leer presupuesto

Leer inversiones

Crear matriz DP

Para i desde 1 hasta número de inversiones

Para j desde 0 hasta presupuesto

Si $\text{costo} \leq j$ entonces
$$\text{DP}[i][j] = \text{máximo} ($$

$$\text{ganancia} + \text{DP}[i-1][j-\text{costo}],$$
$$\text{DP} [i-1] [j]$$

)

Sino

$$\text{DP}[i][j] = \text{DP}[i-1][j]$$

Fin Si

Fin Para

Fin Para

Reconstruir las inversiones seleccionadas

Mostrar inversiones elegidas

Mostrar costo total

Mostrar ganancia total

Fin

3.2. Implementación del Algoritmo

Se enviará el código por un .zip junto con el informe y el informe de medición

IV. RESULTADOS

4.1. Análisis empírico

Funcionalidad	Resultados	Acción
Registros de inversiones	El sistema registro correctamente el nombre, costo y ganancia de cada inversion	Verifico que todos los datos que se ingresaron fueran almacenados correctamente
Validación del presupuesto	Se acepta solo presupuestos mayores a cero	Realizamos pruebas con valores validos e inválidos para que se validan correctamente
Ejecución del algoritmo Voraz	El algoritmo selecciona la inversión que sea mejor en ganancia/costo	Se comparó el resultado obtenido con el esperado para verificar su funcionamiento.
Ejecución de Programación Dinámica (Mochila 0/1)	El algoritmo encontró la combinación de inversiones con la mayor ganancia sin exceder el presupuesto.	Probramos varios casos para así confirmar la mejor solución optima
Comparación entre algoritmos	Se observó que el algoritmo Voraz fue más rápido, mientras que la Programación Dinámica obtuvo mejores resultados en algunos casos.	Se analizaron las diferencias de rendimiento y precisión entre ambos algoritmos.
Presentación de resultados	El sistema mostró las inversiones seleccionadas, el costo total, la ganancia obtenida y el presupuesto restante.	Se verificó que la información presentada fuera clara, completa y correcta.

4.2. Evaluación

Analizar a través de una tabla de doble entrada las ventajas y desventajas de haber utilizado otros algoritmos para solucionar el caso planteado

	Programación Dinámica	Voraz
Ventajas	Encuentra siempre la mejor solución. Evalúa todas las combinaciones necesarias.	Muy rápido y sencillo de implementar.

Desventajas	Consume más memoria y requiere mayor tiempo de procesamiento.	No siempre obtiene la solución óptima.
--------------------	---	--

V. CONCLUSIONES

Mediante las estrategias de programación dinámica y algoritmo voraz, logramos encontrar una solución para el problema que hemos escogido que es sobre la planificación de inversiones con presupuesto limitado, garantizando así una solución óptima en base de la programación en PYTHON que al ser un lenguaje un poco mas sencillo logramos concluir este proyecto con todos los puntos que nos dejó y cumplirlos correctamente.

VIII. REFERENCIAS O BIBLIOGRAFÍA

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms* (4.^a ed.). MIT Press.

Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2014). *Data Structures and Algorithms in Python*. John Wiley & Sons.

Martello, S., & Toth, P. (1990). *Knapsack Problems: Algorithms and Computer Implementations*. John Wiley & Sons.

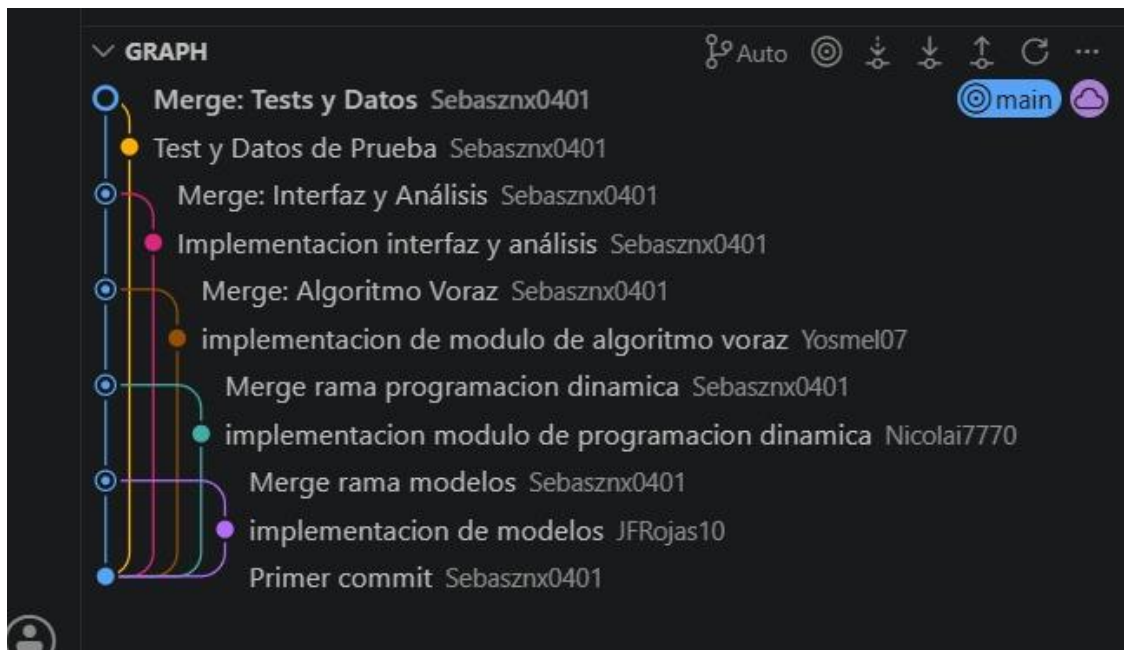
Python Software Foundation. (2026). *Python 3 documentation*.
<https://docs.python.org/3/>

GeeksforGeeks. (2024). *0/1 Knapsack Problem*. <https://www.geeksforgeeks.org/0-1-knapsack-problem-dp-10/>

Investopedia. (2024). *Capital Budgeting*.
<https://www.investopedia.com/terms/c/capitalbudgeting.asp>

GeeksforGeeks. (2024). *Greedy Algorithms*. <https://www.geeksforgeeks.org/greedy-algorithms/>

IX. ANEXOS



```
PS D:\ProyectoFinal_Grupo14_An_AI_Est_Prog> python src/main.py

=====
SISTEMA DE PLANIFICACIÓN DE INVERSIONES CON PRESUPUESTO LIMITADO
Proyecto 14 - Algoritmos Comparativos
=====

-----
MENÚ PRINCIPAL
-----
1. Cargar proyectos desde archivo
2. Cargar proyectos de ejemplo
3. Agregar proyecto manualmente
4. Resolver con Programación Dinámica (DP)
5. Resolver con Algoritmo Voraz (Greedy)
6. Resolver con ambos y comparar
7. Ver análisis de complejidad
8. Ejecutar benchmark
9. Ver proyectos cargados
10. Limpiar proyectos
0. Salir
-----
Selecciona una opción: █
```

```

1  """
2  Módulo de Interfaz de Usuario
3  Proporciona un menú interactivo para utilizar el sistema de planificación
4  """
5
6  import sys
7  from typing import List, Tuple
8+
9+ try:
10+     from tabulate import tabulate
11+ except ModuleNotFoundError: # pragma: no cover - fallback for environments without tabu]
12+     def tabulate(rows, headers=None, tablefmt="simple"):
13+         """Fallback simple table formatter used when tabulate is unavailable."""
14+         if not rows:
15+             return ""
16+
17+         if headers is None:
18+             headers = []
19+
20+         data = [list(map(str, row)) for row in rows]
21+         if headers:
22+             header_row = [str(h) for h in headers]
23+             data = [header_row] + data
24+
25+         widths = []
26+         for col in zip(*data):
27+             widths.append(max(len(str(item)) for item in col))
28+
29+         lines = []
30+         for idx, row in enumerate(data):
31+             cells = [str(item).ljust(widths[col]) for col, item in enumerate(row)]
32+             line = " | ".join(cells)
33+             lines.append(line)
34+             if idx == 0 and headers:
35+                 lines.append("-+-".join("-" * width for width in widths))
36+
37+         return "\n".join(lines)
38+
39  from src.modelos.models import Project, Solution
40  from src.algoritmos.dynamic_programming import solve_knapsack_01
41  from src.algoritmos.greedy_algorithm import solve_greedy, solve_greedy_by_benefit, solve_
42  from src.algoritmos.analyzer import benchmark_algorithms, generate_complexity_report, pri

```

```
from src.modelos.models import Project, Solution
from src.algoritmos.dynamic_programming import solve_knapsack_01
from src.algoritmos.greedy_algorithm import solve_greedy, solve_greedy_by_benefit, s
from src.algoritmos.analyzer import benchmark_algorithms, generate_complexity_report

def print_header():
    """Imprime el encabezado del programa"""
    print("\n" + "="*80)
    print("SISTEMA DE PLANIFICACIÓN DE INVERSIONES CON PRESUPUESTO LIMITADO".center(80))
    print("Proyecto 14 - Algoritmos Comparativos".center(80))
    print("="*80 + "\n")

def print_menu():
    """Imprime el menú principal"""
    print("\n" + "-"*80)
    print("MENÚ PRINCIPAL")
    print("-"*80)
    print("1. Cargar proyectos desde archivo")
    print("2. Cargar proyectos de ejemplo")
    print("3. Agregar proyecto manualmente")
    print("4. Resolver con Programación Dinámica (DP)")
    print("5. Resolver con Algoritmo Voraz (Greedy)")
    print("6. Resolver con ambos y comparar")
    print("7. Ver análisis de complejidad")
    print("8. Ejecutar benchmark")
    print("9. Ver proyectos cargados")
    print("10. Limpiar proyectos")
    print("0. Salir")
    print("-"*80)
```

```
def load_sample_projects() -> List[Project]:
    """
    Carga proyectos de ejemplo para pruebas

    Returns:
        Lista de proyectos de ejemplo
    """
    projects = [
        Project(1, "Software Development Tool", 50000, 80000, 1),
        Project(2, "Mobile App", 30000, 45000, 2),
        Project(3, "Cloud Infrastructure", 40000, 65000, 3),
        Project(4, "Security Update", 15000, 25000, 4),
        Project(5, "Documentation", 10000, 15000, 5),
        Project(6, "API Integration", 20000, 35000, 2),
        Project(7, "Database Optimization", 25000, 40000, 3),
        Project(8, "Testing Framework", 12000, 20000, 4),
    ]
    return projects

def display_projects(projects: List[Project]) -> None:
    """
    Muestra los proyectos en formato tabla

    Args:
        projects: Lista de proyectos a mostrar
    """
    if not projects:
        print("\n⚠ No hay proyectos cargados.")
        return

    print("\n" + "="*100)
    print("PROYECTOS CARGADOS".center(100))
    print("="*100)

    headers = ["ID", "Nombre", "Costo", "Beneficio", "ROI", "Prioridad"]
    data = []

    for p in projects:
        data.append([
```

```
for p in projects:
    data.append([
        p.id,
        p.name[:30], # Truncar nombre si es muy largo
        f"${p.cost:,.0f}",
        f"${p.benefit:,.0f}",
        f"${p.roi():.2f}",
        p.priority
    ])

print(tabulate(data, headers=headers, tablefmt="grid"))
print(f"\nTotal de proyectos: {len(projects)}\n")

def add_project_manual(projects: List[Project]) -> List[Project]:
    """
    Permite agregar un proyecto manualmente

    Args:
        projects: Lista actual de proyectos

    Returns:
        Lista con el nuevo proyecto agregado
    """
    try:
        print("\n" + "-"*80)
        print("AGREGAR PROYECTO MANUALMENTE")
        print("-"*80)

        project_id = len(projects) + 1
        name = input("Nombre del proyecto: ").strip()
        cost = float(input("Costo ($): "))
```

```
try:
    print("\n" + "-"*80)
    print("AGREGAR PROYECTO MANUALMENTE")
    print("-"*80)

    project_id = len(projects) + 1
    name = input("Nombre del proyecto: ").strip()
    cost = float(input("Costo ($): "))
    benefit = float(input("Beneficio esperado ($): "))
    priority = int(input("Prioridad (1-10): "))

    new_project = Project(project_id, name, cost, benefit, priority)
    projects.append(new_project)

    print(f"\n✓ Proyecto agregado exitosamente: {new_project}")
    return projects

except ValueError as e:
    print(f"\nX Error: Entrada inválida. {e}")
    return projects
except Exception as e:
    print(f"\nX Error: {e}")
    return projects
```

```
def get_budget() -> float:
    """
    Solicita el presupuesto disponible al usuario

    Returns:
        Presupuesto como número flotante
    """
    while True:
        try:
            budget = float(input("\nPresupuesto disponible ($): "))
            if budget <= 0:
                print("El presupuesto debe ser positivo.")
                continue
            return budget
        except ValueError:
            print("Por favor, ingresa un número válido.")
```

```
def display_solution(solution: Solution, budget: float) -> None:
    """
    Muestra una solución de forma legible

    Args:
        solution: Solución a mostrar
        budget: Presupuesto disponible
    """
    print("\n" + "="*100)
    print(f"SOLUCIÓN: {solution.algorithm_used}".center(100))
    print("="*100)

    # Información de la solución
    print(f"\nResultados:")
    print(f"  Proyectos seleccionados: {len(solution.projects)}")
    print(f"  Costo total: ${solution.total_cost:,.0f} (Presupuesto: ${budget:,.0f})")
    print(f"  Beneficio total: ${solution.total_benefit:,.0f}")
    print(f"  ROI: {solution.get_roi():.2f}")
    print(f"  Presupuesto utilizado: {(solution.total_cost/budget)*100:.1f}%")
    print(f"  Presupuesto restante: ${budget - solution.total_cost:,.0f}")
```

```
# Tabla de proyectos seleccionados
if solution.projects:
    print(f"\nProyectos seleccionados:")
    headers = ["#", "Nombre", "Costo", "Beneficio", "ROI", "Prioridad"]
    data = []

    for i, p in enumerate(solution.projects, 1):
        data.append([
            i,
            p.name[:40],
            f"${p.cost:,.0f}",
            f"${p.benefit:,.0f}",
            f"{p.roi():.2f}",
            p.priority
        ])

    print(tabulate(data, headers=headers, tablefmt="grid"))
else:
    print("\nNo hay proyectos en la solución.")

print("="*100 + "\n")
```



```
def display_comparison(dp_solution: Solution, greedy_solution: Solution, budget: float) -> None:
    """
    Muestra una comparación lado a lado de dos soluciones

    Args:
        dp_solution: Solución de Programación Dinámica
        greedy_solution: Solución del Algoritmo Voraz
        budget: Presupuesto disponible
    """

    print("\n" + "="*100)
    print("COMPARACIÓN DE SOLUCIONES".center(100))
    print("="*100)

    # Tabla comparativa
    comparison_data = [
        ["Algoritmo", "DP (Óptimo)", "Voraz (Heurística)"],
        ["Proyectos", len(dp_solution.projects), len(greedy_solution.projects)],
        ["Costo total", f"${dp_solution.total_cost:.0f}", f"${greedy_solution.total_cost:.0f}"],
        ["Beneficio total", f"${dp_solution.total_benefit:.0f}", f"${greedy_solution.total_benefit:.0f}"],
        ["ROI", f"${dp_solution.get_roi():.2f}", f"${greedy_solution.get_roi():.2f}"],
        ["% Presupuesto usado", f"${(dp_solution.total_cost/budget)*100:.1f}%",
        f"${(greedy_solution.total_cost/budget)*100:.1f}%"],
    ]
```

```
print()
for row in comparison_data:
    print(f" {row[0]:.<25} {row[1]:>30} | {row[2]:>30}")

# Análisis
dp_better = dp_solution.total_benefit > greedy_solution.total_benefit
difference = abs(dp_solution.total_benefit - greedy_solution.total_benefit)
optimality = (greedy_solution.total_benefit / dp_solution.total_benefit * 100) \
    if dp_solution.total_benefit > 0 else 100

print(f"\n {'ANÁLISIS':.<25}")
print(f" {'Diferencia en beneficio':.<25} ${difference:.0f}")
print(f" {'Optimalidad de Voraz':.<25} {optimality:.1f}%")

if dp_better:
    print(f" {'Recomendación':.<25} DP produce {optimality:.1f}% más beneficio")
else:
    print(f" {'Recomendación':.<25} Ambas dan resultados similares")

print("="*100 + "\n")
```

```
def main():
    """Función principal - Menú interactivo"""
    print_header()

    projects: List[Project] = []
    budget: float = 0

    while True:
        try:
            print_menu()
            choice = input("Selecciona una opción: ").strip()

            if choice == "1":
                # Cargar desde archivo
                print("\n⚠ Opción no implementada en esta versión.")
                print("Por favor, usa la opción 2 (proyectos de ejemplo) o 3 (agregar manualmente).")

            elif choice == "2":
                # Cargar proyectos de ejemplo
                projects = load_sample_projects()
                print(f"\n✓ {len(projects)} proyectos de ejemplo cargados.")
                display_projects(projects)

            elif choice == "3":
                # Agregar proyecto manualmente
                if not projects:
                    print("\nPrimero debe cargar proyectos (opción 2) o agregar uno (opción 3).")
                projects = add_project_manual(projects)

            elif choice == "4":
                # Resolver con DP
                if not projects:
```

```
                    elif choice == "4":
                        # Resolver con DP
                        if not projects:
                            print("\nX Error: No hay proyectos cargados.")
                            continue
                        budget = get_budget()
                        print("\n⌚ Resolviendo con Programación Dinámica...")
                        solution = solve_knapsack_01(projects, budget)
                        display_solution(solution, budget)

                    elif choice == "5":
                        # Resolver con Voraz
                        if not projects:
                            print("\nX Error: No hay proyectos cargados.")
                            continue
                        budget = get_budget()
                        print("\n⌚ Resolviendo con Algoritmo Voraz...")
                        solution = solve_greedy(projects, budget)
                        display_solution(solution, budget)

                    elif choice == "6":
                        # Comparar ambos
                        if not projects:
                            print("\nX Error: No hay proyectos cargados.")
                            continue
                        budget = get_budget()
                        print("\n⌚ Resolviendo con ambos algoritmos...")
                        dp_solution = solve_knapsack_01(projects, budget)
                        greedy_solution = solve_greedy(projects, budget)
```

```

dp_solution = solve_knapsack_01(projects, budget)
greedy_solution = solve_greedy(projects, budget)
display_solution(dp_solution, budget)
display_solution(greedy_solution, budget)
display_comparison(dp_solution, greedy_solution, budget)

elif choice == "7":
    # Análisis de complejidad
    print(generate_complexity_report())

elif choice == "8":
    # Benchmark
    if not projects:
        print("\nX Error: No hay proyectos cargados.")
        continue
    budget = get_budget()
    print("\nX Ejecutando benchmark...")
    results = benchmark_algorithms(projects, budget)
    print_benchmark_results(results)

elif choice == "9":
    # Ver proyectos
    display_projects(projects)

elif choice == "10":
    # Limpiar proyectos
    projects = []
    print("\n✓ Proyectos eliminados.")

```

```

elif choice == "0":
    # Salir
    print("\n¡Hasta luego! 🙌")
    sys.exit(0)

else:
    print("\nX Opción no válida. Por favor, intenta de nuevo.")

except KeyboardInterrupt:
    print("\n\n¡Programa interrumpido por el usuario!")
    sys.exit(0)
except Exception as e:
    print(f"\nX Error inesperado: {e}")
    import traceback
    traceback.print_exc()

if __name__ == "__main__":
    main()

```

```

MENÚ PRINCIPAL
-----
1. Cargar proyectos desde archivo
2. Cargar proyectos de ejemplo
3. Agregar proyecto manualmente
4. Resolver con Programación Dinámica (DP)
5. Resolver con Algoritmo Voraz (Greedy)
6. Resolver con ambos y comparar
7. Ver análisis de complejidad
8. Ejecutar benchmark
9. Ver proyectos cargados
10. Limpiar proyectos
0. Salir
-----
Selecciona una opción: 4

Presupuesto disponible ($): 25000

🕒 Resolviendo con Programación Dinámica...
  
```

```

=====
                        SOLUCIÓN: Dynamic Programming (Mochila 0/1)
=====

Resultados:
  Proyectos seleccionados: 2
  Costo total: $25,000 (Presupuesto: $25,000)
  Beneficio total: $40,000
  ROI: 1.60
  Presupuesto utilizado: 100.0%
  Presupuesto restante: $0

Proyectos seleccionados:
+-----+-----+-----+-----+-----+
| # | Nombre       | Costo  | Beneficio | ROI   | Prioridad |
+-----+-----+-----+-----+-----+
| 1 | Documentation | $10,000 | $15,000   | 1.5   | 5         |
+-----+-----+-----+-----+-----+
| 2 | Security Update | $15,000 | $25,000   | 1.67  | 4         |
+-----+-----+-----+-----+-----+
  
```

https://github.com/Sebaszrx0401/ProyectoFinal_Grupo14_An_AI_Est_Prog

<https://www.youtube.com/watch?v=2PUKCH75Aoc&feature=youtu.be>

NOTAS ACLARATORIAS

1. El Informe de proyecto de fin de curso será redactado con procesador de textos en fuente Arial, tamaño 11, alineación justificada, interlineado 1,5, hoja A4. Utilizar formato APA 7ma edición.
2. Sobre el tema de proyecto
 - ✓ Integración con el aprendizaje previo (temas aprendidos en otros cursos)
 - ✓ Promueve un nuevo aprendizaje
 - ✓ Brinda experiencias realistas

- ✓ Balance entre complejidad y carga de trabajo
- 3. Comunicación
 - ✓ Lenguaje profesional
 - ✓ Organización de la presentación
 - ✓ Presentación efectiva
 - ✓ Figuras y formato
 - ✓ Redacción y gramática